Github 管理

我(管理者)要先：

```
git init
git add .
git commit -m "Initial commit: 上傳實驗基礎架構"
git branch -M main
git remote add origin <你的 GitHub Repo 網址>
git push -u origin main
```

大家要：

為了讓大家知道目前要做什麼，可以使用 Issue 來管理任務。

1. **建立 Issue：** 當有新功能要寫（例如：實作多工器）或是發現 Bug 時，到專案的 **Issues** 標籤頁點擊「New issue」。
2. **描述任務：** 寫下這個任務的目標，並可以 Assign 給特定的同學。每個 Issue 都會有一個專屬編號（例如 `#1`）。

你們會遇到以下幾種情況：

1. 每次要開發一個新功能前：
   a. 開始工作前，一定要抓最新架構

   ```
   git checkout main
   git pull origin main
   ```

   b. 請去發Issue告訴別人你要做什麼，然後開新分支

   ```
   git checkout -b feature/xxx
   ```

   c. 努力寫Code
   d. 存檔、提交、推送到自己的分支

   ```
   git add .
   ```

```
git commit -m "xxx is completed."
git push origin feature/xxx
```

e. 去Github 按下去發Pull Request, 要對應到 Issue上面的序號

2. 開發到一半被彎道超車，你當初pull的main 不是最新的main了
   a. 確認自己在目前開發的分支上 feature/xxx
   b. 將遠端main最新進度拉到當前分支

```
git pull origin main
```

   c. 繼續開發，正常Push，PR就會包含最新的main的修改。

3. 你們Merge conflict了
   a. 同學A 與同學B 修改了同一個區塊
   b. 這時同學A 執行 git pull origin main 終端機會顯示 Merge conflict
   c. Git 會要求A手動決定要保留誰的程式碼

```
<<<<<<< HEAD
wire [3:0] adder_out; // 這是同學 A 自己寫的程式碼
=======
wire [6:0] display_out; // 這是同學 B 寫的程式碼，同學 B 的程式碼已經存在
>>>>>>> origin/main
```

   d. 你需要手動把 <<< === >>> 都刪掉，其他衝突的地方可以刪減或都保留
   e. 告訴git 衝突已解決

```
git add top.v
git commit -m "同學 A 解決了 top.v 的合併衝突"
git push origin feature/xxx
```

4. 你們發的PR因為錯誤被我退回去了
   a. 不要把這個PR關掉，也不用開新的分支
   b. 直接在本地端的同一個分支 feature/xxx 修正
   c. 再次存檔提交

```
git add .
git commit -m "修正訊號線接反的問題"
git push origin feature/adder
```

d. 在畫面右側的 **Reviewers** 欄位找到我的名字，旁邊會有一個由兩個箭頭組成的重新整理小圖示（Re-request review）。點擊它，GitHub 就會發送通知給我(管理員)。

5. 更多情況請先**告知全部的人然後想出解決方法**

管理者要：

1. 當有人發PR進來
   a. 更新本地端資訊，讓我的Git知道Github上有同學的新分支

```
git fetch origin
```

   b. 切換到同學的分支（記住我還沒合併他，但他是節外生枝，我可以測試他）

```
git checkout feature/xxx
```

   c. 在本地端測試
      i. 測試通過 → 回終端機切回 git checkout main。接著去Github 按下 Merge PR
      ii. 測試失敗 → 一樣切回main。去Github按下Request Changes退回給他